

# WORKSHOP

## PILE D'EXÉCUTION

NUCLEO-L476RG · STM32L476RG · Cortex-M4

VSCode · PlatformIO · Cortex-Debug · ST-Link

## Ce que vous allez apprendre dans ce workshop

- Comprendre comment la mémoire est organisée sur un microcontrôleur STM32
- Observer en temps réel l'état de la pile lors de l'exécution d'un programme C
- Maîtriser le débogueur intégré de VSCode (breakpoints, step-by-step, CALL STACK, REGISTERS)
- Mesurer et analyser la taille de la pile à l'aide des outils de développement
- Comprendre le lien entre l'appel de fonction en C et ce qui se passe réellement en mémoire

**Durée : 3h30 | Niveau : débutant à intermédiaire**

## Table des matières

| Section                        | Contenu                                                 | Durée  |
|--------------------------------|---------------------------------------------------------|--------|
| 1 — Introduction               | Concepts clés : pile, mémoire, microcontrôleur STM32    | 15 min |
| 2 — Installation               | VSCode, PlatformIO, Cortex-Debug, droits ST-Link        | 25 min |
| 3 — Première compilation       | Créer un projet, écrire le premier programme, compiler  | 35 min |
| 4 — Prise en main du débogueur | ST-Link, breakpoints, CALL STACK, REGISTERS, pas-à-pas  | 35 min |
| 5 — Exercice mémoire           | Analyser les sections du firmware, linker script        | 30 min |
| 6 — Exercice pile              | Mesurer SP, observer la pile lors d'appels de fonctions | 50 min |

## 1 — Introduction

### Comprendre la pile avant de l'observer

Dans ce workshop, vous allez découvrir de l'intérieur ce qui se passe en mémoire quand un programme s'exécute sur un microcontrôleur STM32. Vous allez observer en direct — via un débogueur professionnel — comment les fonctions s'empilent et se dépilent, où vivent les variables, et combien de mémoire consomme réellement votre code.

#### 1.1 La carte NUCLEO-L476RG

La carte que vous utilisez est une carte de développement professionnelle fabriquée par STMicroelectronics. Elle héberge le microcontrôleur STM32L476RG — un circuit intégré qui contient sur une seule puce un processeur ARM Cortex-M4, de la mémoire Flash, de la mémoire SRAM, et de nombreux périphériques.

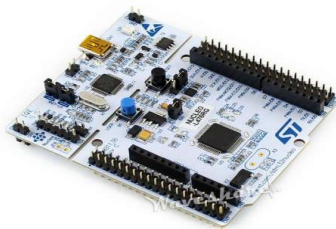


Figure 1 — Carte NUCLEO-L476RG (source : Waveshare / STMicroelectronics)

| Ressource          | Valeur                                                     |
|--------------------|------------------------------------------------------------|
| Processeur         | ARM Cortex-M4F — 80 MHz — FPU (virgule flottante) intégrée |
| Flash              | 1 024 Ko — stocke le programme et les constantes           |
| SRAM totale        | 96 Ko — SRAM1 (64 Ko) + SRAM2 (32 Ko)                      |
| Pile (stack)       | 1 Ko par défaut (configurable dans le linker script)       |
| Tas (heap)         | 512 octets par défaut (configurable dans le linker script) |
| Interface debug    | ST-Link v2 intégrée — débogage professionnel via SWD       |
| LED utilisateur    | LD2 verte — broche PA5                                     |
| Bouton utilisateur | B1 bleu — broche PC13                                      |

*Note pour les étudiants : La carte NUCLEO est conçue pour l'apprentissage et le prototypage. Le ST-Link intégré signifie que vous n'avez besoin que d'un câble USB pour programmer et déboguer — aucun matériel supplémentaire n'est nécessaire.*

## 1.2 Qu'est-ce que la pile d'exécution ?

La pile (ou stack en anglais) est une zone de la mémoire SRAM utilisée automatiquement par le processeur à chaque appel de fonction. Elle fonctionne comme une pile d'assiettes : on empile en haut, on prend toujours depuis le haut. En informatique, on appelle ça LIFO — Last In, First Out (dernier entré, premier sorti).

### Évolution de la pile lors d'appels imbriqués

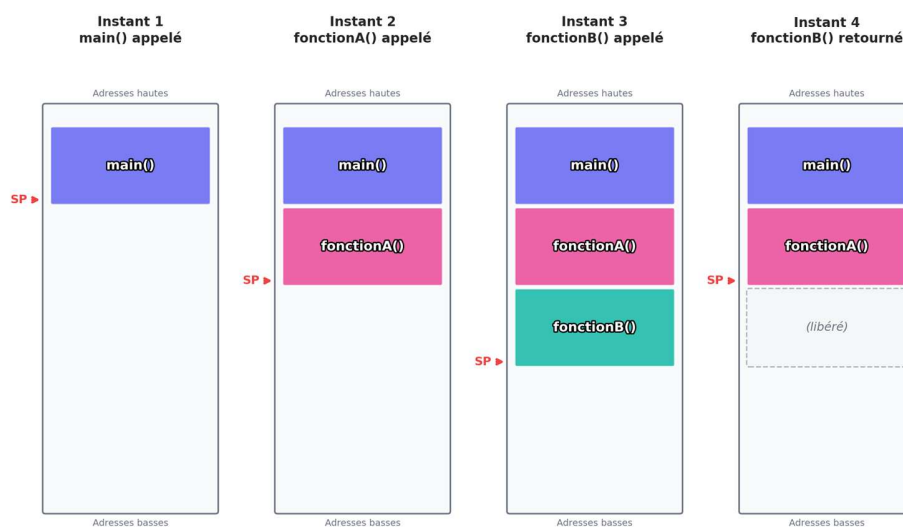


Figure 2 — Évolution de la pile lors d'appels de fonctions imbriqués

### Ce que le processeur empile à chaque appel de fonction :

- L'adresse de retour — pour savoir où reprendre l'exécution quand la fonction se termine. Stockée dans le registre LR (Link Register) par l'instruction BL (Branch with Link), puis sauvegardée sur la pile si la fonction appelle elle-même d'autres fonctions.
- Les paramètres passés à la fonction — si les 4 premiers tiennent dans R0-R3, ils y restent (convention ARM AAPCS). Les paramètres supplémentaires sont empilés.
- Les variables locales de la fonction — déclarées dans le corps de la fonction, elles vivent sur la pile.
- Les registres callee-saved (R4-R11) — si la fonction les utilise, elle doit les sauvegarder sur la pile puis les restaurer avant de retourner. C'est le compilateur qui génère ces instructions automatiquement.

**ATTENTION — Ne pas confondre !** L'empilement automatique de 8 registres (R0-R3, R12, LR, PC, xPSR = 32 octets) ne se produit que lors d'une INTERRUPTION (exception matérielle), PAS lors d'un appel de fonction normal via BL. Dans un appel BL, seul LR reçoit l'adresse de retour — c'est la fonction appelée qui gère la sauvegarde des registres qu'elle utilise.

L'ensemble de ces informations pour un appel de fonction s'appelle un frame (ou cadre de pile). Quand la fonction se termine, son frame est dépilé et la mémoire qu'elle occupait est libérée automatiquement.

### 1.3 Schéma : évolution de la pile lors d'un appel de fonction

```
/* Programme : main() appelle fonctionA(), qui appelle fonctionB() */
```

SRAM – vue de la pile à différents instants :

Instant 1 : seul main() tourne

|            |              |                              |
|------------|--------------|------------------------------|
| 0x20018000 | +-----+      | <- SP initial (haut de pile) |
|            | frame main() | variables locales de main    |
| SP ->      | +-----+      |                              |

Instant 2 : main() appelle fonctionA()

|            |                   |                                                |
|------------|-------------------|------------------------------------------------|
| 0x20018000 | +-----+           |                                                |
|            | frame main()      |                                                |
|            | +-----+           |                                                |
|            | adresse retour    | <- le processeur sait où reprendre dans main() |
|            | frame fonctionA() | variables locales de fonctionA                 |
| SP ->      | +-----+           | <- SP a diminué (la pile grandit vers le BAS)  |

Instant 3 : fonctionA() appelle fonctionB()

|            |                   |                                  |
|------------|-------------------|----------------------------------|
| 0x20018000 | +-----+           |                                  |
|            | frame main()      |                                  |
|            | +-----+           |                                  |
|            | frame fonctionA() |                                  |
|            | +-----+           |                                  |
|            | adresse retour    | <- où reprendre dans fonctionA() |
|            | frame fonctionB() |                                  |
| SP ->      | +-----+           | <- SP au plus bas                |

Instant 4 : fonctionB() se termine, son frame est dépilé  
SP remonte automatiquement -> on retrouve l'état de l'instant 2

*Analogie pour mieux comprendre : Imaginez une pile de plateaux à la cantine. Vous ne pouvez prendre que le plateau du dessus (le dernier posé). Quand une fonction est appelée, un nouveau plateau est posé. Quand elle se termine, le plateau est retiré. Si vous empilez trop de plateaux, la pile s'effondre — c'est un stack overflow.*

## A retenir — Pile d'exécution

- SP (Stack Pointer) = registre du processeur qui pointe toujours vers le sommet de la pile.
- Sur Cortex-M4, la pile est descendante : SP diminue quand on empile, augmente quand on dépile.
- La taille de la pile utilisée à un instant  $t$  = adresse initiale de SP - valeur courante de SP.
- Une pile qui déborde (stack overflow) provoque un crash du programme (HardFault sur STM32).

## 2 — Installation des prérequis

**VSCode · PlatformIO · Cortex-Debug · ST-Link**

*Note : Les commandes d'installation ci-dessous sont pour Linux (Ubuntu/Debian). Si vous travaillez sous Windows, VSCode se télécharge depuis le site officiel, et PlatformIO + Cortex-Debug s'installent de la même façon via l'interface d'extensions VSCode. La section 2.4 (règles udev) ne s'applique qu'à Linux.*

| Outil                     | Rôle                                                                                            |
|---------------------------|-------------------------------------------------------------------------------------------------|
| <b>VSCode</b>             | Éditeur de code — interface principale du workshop                                              |
| <b>PlatformIO IDE</b>     | Extension VSCode — gère la compilation, le téléversement et le débogage pour STM32              |
| <b>Cortex-Debug</b>       | Extension VSCode — interface graphique du débogueur ARM (breakpoints, CALL STACK, REGISTERS...) |
| <b>OpenOCD</b>            | Installé automatiquement par PlatformIO — fait le lien entre VSCode et le ST-Link               |
| <b>arm-none-eabi-gcc</b>  | Installé automatiquement par PlatformIO — compilateur C pour ARM Cortex-M4                      |
| <b>ST-Link (matériel)</b> | Intégré sur la carte NUCLEO-L476RG — pas besoin d'acheter quoi que ce soit                      |

### 2.1 Installer Visual Studio Code

Ouvrir un terminal (Ctrl+Alt+T) et exécuter les commandes suivantes une par une :

```
# Etape 1 – Télécharger le paquet d'installation .deb (Linux uniquement)
wget -O /tmp/vscode.deb
"https://code.visualstudio.com/sha/download?build=stable&os=linux-deb-x64"

# Etape 2 – Installer VSCode
sudo dpkg -i /tmp/vscode.deb

# Etape 3 – Corriger les eventuelles dependances manquantes
sudo apt-get install -f -y
```

```
# Etape 4 – Lancer VSCode pour vérifier  
code
```

*Si la commande wget échoue : Vérifier la connexion internet (ping google.com). Sous Windows : télécharger l'installateur .exe depuis <https://code.visualstudio.com> et suivre l'assistant d'installation.*

## 2.2 Installer l'extension PlatformIO IDE

PlatformIO est l'environnement de développement pour systèmes embarqués. Il installe automatiquement tous les outils nécessaires (compilateur ARM, bibliothèques HAL, OpenOCD).

1. Dans VSCode, appuyer sur Ctrl+Shift+X pour ouvrir le gestionnaire d'extensions
2. Dans la barre de recherche, taper : PlatformIO IDE
3. Cliquer sur l'extension publiée par "PlatformIO"
4. Cliquer sur le bouton bleu "Install"
5. Attendre 3 à 5 minutes — PlatformIO installe Python, pip et ses dépendances
6. Cliquer sur "Reload Now" quand le message apparaît

**Signe de succès :** une icône fourmi apparaît dans la barre latérale gauche de VSCode.

## 2.3 Installer l'extension Cortex-Debug

Cortex-Debug est l'extension qui fournit l'interface graphique du débogueur dans VSCode : visualisation de la pile, des registres, des variables, des périphériques STM32.

1. Ctrl+Shift+X -> rechercher : Cortex-Debug
2. Sélectionner l'extension publiée par "marus25"
3. Cliquer sur "Install"

## 2.4 Configurer les droits d'accès au ST-Link (Linux uniquement)

*Windows : Les drivers ST-Link sont installés automatiquement. Si la carte n'est pas reconnue, installer les drivers ST-Link depuis le site STMicroelectronics (STSW-LINK009).*

Sous Linux, l'accès aux périphériques USB comme le ST-Link nécessite des règles de permissions spéciales (règles udev). Sans elles, VSCode ne peut pas communiquer avec la carte.

```
# Ajouter les règles de permissions pour PlatformIO / ST-Link  
curl -fsSL https://raw.githubusercontent.com/platformio/platformio-  
core/master/scripts/99-platformio-udev.rules \  
| sudo tee /etc/udev/rules.d/99-platformio-udev.rules  
  
# Appliquer les nouvelles règles  
sudo service udev restart
```

```
# Ajouter l'utilisateur courant aux groupes nécessaires
sudo usermod -a -G dialout $USER
sudo usermod -a -G plugdev $USER
```

**Action obligatoire après ces commandes :** Se déconnecter (logout) puis se reconnecter — les nouveaux droits de groupe ne sont actifs qu'après reconnexion. Débrancher et rebrancher le câble USB de la carte NUCLEO.

*Dans un environnement de machine virtuelle (VirtualBox / VMware) : Brancher la carte NUCLEO-L476RG au PC hôte via USB. Dans le menu de la VM -> Périphériques -> USB -> cocher "STMicroelectronics STLink". Vérifier avec : `lsusb | grep -i stlink`*

## 3 — Première compilation

### 3.1 Créer un nouveau projet PlatformIO

Procédure pas-à-pas :

1. Cliquer sur l'icône fourmi dans la barre latérale gauche -> "PIO Home" s'ouvre
2. Cliquer sur le bouton "+ New Project"
3. Remplir les champs (voir tableau ci-dessous)
4. Cliquer sur "Finish"
5. Attendre — PlatformIO génère la structure du projet

| Champ     | Valeur a saisir                                          |
|-----------|----------------------------------------------------------|
| Name      | workshop_pile (pas d'espace ni de caractère spécial)     |
| Board     | Taper "nucleo_l476rg" -> sélectionner "ST Nucleo L476RG" |
| Framework | STM32Cube                                                |
| Location  | Laisser le chemin par défaut                             |

### 3.2 Structure du projet — comprendre les fichiers

```
workshop_pile/
|
+-- src/
|   +-- main.c           <- VOTRE CODE SOURCE — c'est ici que vous travaillez
|
+-- include/             <- vos fichiers .h personnels (vide pour l'instant)
|
+-- lib/                 <- bibliothèques locales au projet (vide pour l'instant)
```

```
|
+-- .pio/                                <- généré automatiquement a la compilation (ne pas
modifier)
|   +-- build/
|       +-- nucleo_l476rg/
|           +-- firmware.elf  <- binaire avec symboles de debug (utilise par le
débogueur)
|               +-- firmware.bin  <- binaire brut (téléversé dans la Flash du STM32)
|
+-- platformio.ini                        <- configuration du projet (carte, framework, options)
```

*Pour les étudiants : Vous ne modifierez que deux fichiers pendant ce workshop : src/main.c (votre code C) et platformio.ini (la configuration). Tout le reste est géré automatiquement par PlatformIO.*

#### Contenu initial de platformio.ini :

```
[env:nucleo_l476rg]
platform = ststm32      ; famille de cartes STM32
board     = nucleo_l476rg ; modèle exact de la carte
framework = stm32cube   ; HAL officiel de STMicroelectronics
```

### 3.3 Premier programme — LED LD2 clignotante

Ouvrir src/main.c dans l'explorateur. Si le fichier est vide ou contient du contenu généré par défaut, sélectionner tout (Ctrl+A), supprimer, et coller le code complet ci-dessous.

#### Avant de coller le code — comprendre ce qu'il fait :

| Ligne / fonction            | Ce qu'elle fait                                                              |
|-----------------------------|------------------------------------------------------------------------------|
| <b>HAL_Init()</b>           | Initialise la bibliothèque HAL et configure le SysTick (timer 1 ms)          |
| <b>SystemClock_Config()</b> | Configure l'oscillateur interne (HSI 16 MHz) et la PLL pour atteindre 80 MHz |
| <b>MX_GPIO_Init()</b>       | Active l'horloge du port GPIOA et configure la broche PA5 en sortie          |
| <b>HAL_GPIO_WritePin()</b>  | Met PA5 à 3,3V (LED allumée) ou à 0V (LED éteinte)                           |
| <b>HAL_Delay(1000)</b>      | Attend 1000 ms — fonctionne grâce au SysTick configuré dans HAL_Init()       |
| <b>SysTick_Handler()</b>    | Appelé toutes les 1 ms par le timer — incrémente le compteur interne du HAL  |

#### Programme 1 — Code complet a copier dans src/main.c :

```
/**
 * Workshop Pile d'exécution — NUCLEO-L476RG
 * Programme 1 : LED LD2 (PA5) clignotante
 */
```

- \* OBJECTIF PEDAGOGIQUE :
- \* Ce premier programme vous familiarise avec la structure d'un
- \* programme bare-metal (sans OS) sur STM32. Vous apprendrez le
- \* schéma d'initialisation standard : HAL -> Horloge -> GPIO -> Boucle.
- \*
- \* FONCTIONNEMENT :
- \* La LED LD2 verte est câblée sur la broche PA5 du STM32L476RG.

### Carte NUCLEO-L476RG — Composants principaux

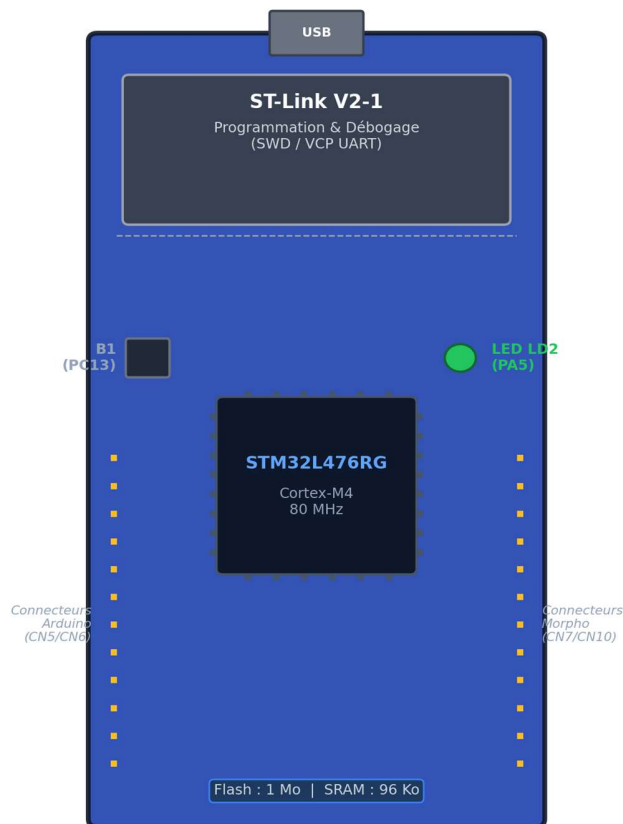


Figure 3 — Composants principaux de la carte NUCLEO-L476RG

- \* Elle s'allume quand PA5 = 3,3V (HIGH) et s'éteint quand PA5 = 0V (LOW).
- \*
- \* IMPORTANT : SysTick\_Handler() DOIT être défini dans ce fichier.
- \* Sans lui, HAL\_Delay() ne fonctionne pas car le compteur de
- \* millisecondes (uwTick) n'est jamais incrémenté -> le programme
- \* reste bloqué indéfiniment dans HAL\_Delay().
- \*
- \* SCHEMA D'EXECUTION (ce qui se passe en mémoire) :

```
*
*   Reset_Handler()          <- code de démarrage STM32
*   +-- main()                <- votre point d'entrée
*   +-- HAL_Init()            <- configure SysTick (interruption chaque 1 ms)
*   +-- SystemClock_Config() <- PLL : 16 MHz HSI -> 80 MHz SYSCLK
*   +-- MX_GPIO_Init() <- PA5 en sortie push-pull
*   +-- while(1)              <- boucle infinie (le programme ne s'arrête jamais)
*   +-- WritePin(SET)         -> LED ON
*   +-- Delay(1000)           -> attend 1 seconde
*   +-- WritePin(RESET)      -> LED OFF
*   +-- Delay(1000)           -> attend 1 seconde
*/

#include "stm32l4xx_hal.h"

/* DECLARATIONS ANTICIPEES (forward déclarations)
 * Le compilateur C lit le fichier de haut en bas. Ces déclarations
 * lui indiquent que ces fonctions existent – leur corps est défini
 * plus bas dans le fichier. */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);

int main(void) {
    /* ETAPE 1 : Initialiser le HAL
     * HAL_Init() fait 3 choses essentielles :
     * 1. Configure le SysTick pour générer une interruption toutes les 1 ms
     * 2. Initialise le compteur uwTick à 0 (utilise par HAL_Delay)
     * 3. Configure la priorité des interruptions (NVIC)
     * DOIT être appelée EN PREMIER, avant toute autre fonction HAL. */
    HAL_Init();

    /* ETAPE 2 : Configurer l'horloge système à 80 MHz
     * Calcul PLL : SYSCLK = (HSI / PLLM) x PLLN / PLLR
     *              = (16 MHz / 1) x 10 / 2 = 80 MHz */
    SystemClock_Config();

    /* ETAPE 3 : Configurer la broche GPIO pour la LED
     * POINT CLE : Sur STM32, chaque périphérique a une horloge dédiée
     * qui est désactivée par défaut. Il faut TOUJOURS activer l'horloge
     * d'un port GPIO avant de l'utiliser. */
    MX_GPIO_Init();

    /* BOUCLE PRINCIPALE : clignoter indéfiniment
     * Sur un microcontrôleur, le programme ne doit JAMAIS se terminer.
     * La boucle while(1) garantit que le programme tourne pour toujours. */
```

```
while (1) {
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);    /* LED ON */
    HAL_Delay(1000);                                         /* 1 sec */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);  /* LED OFF */
    HAL_Delay(1000);                                         /* 1 sec */
}

/* Configuration de l'horloge : HSI 16 MHz -> PLL -> SYSCLK 80 MHz
 * Formule : SYSCLK = (HSI / PLLM) x PLLN / PLLR
 *          = (16 / 1) x 10 / 2 = 80 MHz
 * FLASH_LATENCY_4 : à 80 MHz, le CPU doit attendre 4 cycles
 * pour chaque lecture Flash (la Flash est plus lente que le CPU). */
void SystemClock_Config(void) {
    RCC_OscInitTypeDef RCC_OscInitStruct = {0}; /* {0} = initialiser a zero */
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

    RCC_OscInitStruct.OscillatorType      = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISetup            = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState        = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource       = RCC_PLLSOURCE_HSI; /* Source = HSI 16
MHz */
    RCC_OscInitStruct.PLL.PLLM            = 1;    /* /1 -> 16 MHz */
    RCC_OscInitStruct.PLL.PLLN            = 10;   /* x10 -> 160 MHz (VCO) */
    RCC_OscInitStruct.PLL.PLLR            = RCC_PLLR_DIV2; /* /2 -> 80 MHz */
    HAL_RCC_OscConfig(&RCC_OscInitStruct);

    RCC_ClkInitStruct.ClockType            = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
                                             | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYSCLKSource         = RCC_SYSCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider        = RCC_SYSCLK_DIV1;    /* HCLK = 80 MHz */
    RCC_ClkInitStruct.APB1CLKDivider       = RCC_HCLK_DIV1;     /* APB1 = 80 MHz */
    RCC_ClkInitStruct.APB2CLKDivider       = RCC_HCLK_DIV1;     /* APB2 = 80 MHz */
    HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4);
}

/* Configuration GPIO : PA5 en sortie push-pull pour la LED LD2
 * Push-pull : la broche peut fournir du courant (3,3V) ET absorber (0V)
 * GPIO_SPEED_FREQ_LOW : suffisant pour une LED (1 Hz) */
static void MX_GPIO_Init(void) {
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    __HAL_RCC_GPIOA_CLK_ENABLE(); /* CRITIQUE : activer l'horloge GPIOA */
    HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET); /* LED eteinte */
    GPIO_InitStruct.Pin    = GPIO_PIN_5;
```

```

    GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP; /* Sortie push-pull */
    GPIO_InitStruct.Pull = GPIO_NOPULL;
    GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}

/* HANDLERS D'INTERRUPTION
 * Ces fonctions sont appelées par le matériel via la TABLE DE VECTEURS.
 * Quand une interruption se déclenche, le Cortex-M4 :
 * 1. Empile automatiquement R0-R3, R12, LR, PC, xPSR (32 octets)
 * 2. Lit l'adresse du handler dans la table de vecteurs
 * 3. Saute a cette adresse
 * C'est ici (et UNIQUEMENT lors d'interruptions) que l'empilement
 * automatique de 32 octets se produit – pas lors d'appels BL normaux. */

/* SysTick_Handler : appelle toutes les 1 ms par le timer SysTick
 * SANS cette fonction -> HAL_Delay() boucle indéfiniment */
void SysTick_Handler(void) { HAL_IncTick(); }

/* NMI_Handler : Non-Maskable Interrupt */
void NMI_Handler(void) { }

/* HardFault_Handler : erreur fatale (accès mémoire invalide,
 * stack overflow, division par zéro...) */
void HardFault_Handler(void) { while (1) { } }

```

### 3.4 Compiler et téléverser le programme

| Action              | Comment faire                                                                 |
|---------------------|-------------------------------------------------------------------------------|
| Compiler (Build)    | Ctrl+Alt+B ou icône en bas de VSCode ou terminal : pio run                    |
| Téléverser (Upload) | Ctrl+Alt+U ou icône -> en bas de VSCode ou terminal : pio run --target upload |
| Compiler + Upload   | Icône -> directement (compile si nécessaire avant d'envoyer)                  |

#### Résultat attendu après compilation réussie :

```

Compiling .pio/build/nucleo_l476rg/src/main.o
Linking .pio/build/nucleo_l476rg/firmware.elf
Building .pio/build/nucleo_l476rg/firmware.bin

RAM: [          ] 1.2% (used 1176 bytes from 98304 bytes)
Flash: [          ] 2.1% (used 21504 bytes from 1048576 bytes)
===== [SUCCESS] =====

```

*Comprendre le résultat : RAM 1.2% = votre programme n'utilise que 1 176 octets de SRAM sur les 96 Ko disponibles (variables globales seulement — la pile n'est pas comptée ici). Flash 2.1% = le code compile occupe ~21 Ko sur 1 Mo de Flash.*

**La LED LD2 verte sur la carte clignote à 1 Hz (1 seconde allumée, 1 seconde éteinte).**

**Si la compilation échoue :**

- "stm32l4xx\_hal.h: No such file or directory" -> le HAL n'a pas été téléchargé. Vérifier la connexion internet.
- "Error: LIBUSB\_ERROR\_ACCESS" -> les règles udev ne sont pas appliquées (Linux). Vérifier la déconnexion/reconnexion.
- "Error: open failed" -> la carte n'est pas reconnue. Vérifier le branchement USB ou la redirection VM.
- LED ne clignote pas après upload -> vérifier que SysTick\_Handler() est bien présent dans main.c.

## 4 — Prise en main du débogueur

**Breakpoints · CALL STACK · REGISTERS · Pas-à-pas**

La carte NUCLEO-L476RG intègre une sonde de débogage matérielle appelée ST-Link v2. Elle communique avec le STM32L476RG via le protocole SWD (Serial Wire Debug) — seulement 2 fils — et permet un débogage complet sans ralentir ni modifier le programme.

**Schéma de la chaîne de débogage :**

### Chaîne de débogage STM32

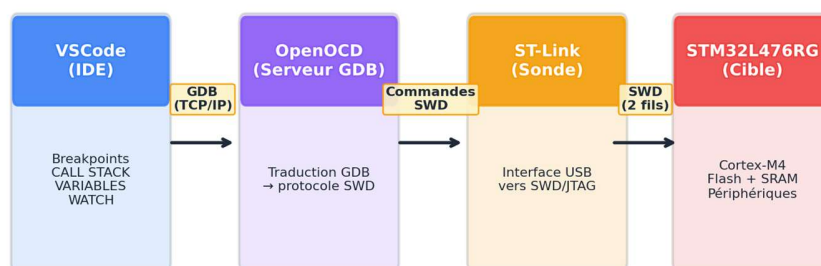


Figure 4 — Chaîne de débogage : VSCoDe -> OpenOCD -> ST-Link -> STM32

|                                    |                                                          |                                          |                                  |
|------------------------------------|----------------------------------------------------------|------------------------------------------|----------------------------------|
| VSCode<br>(Interface<br>graphique) | OpenOCD<br>(pont logiciel<br>installé par<br>PlatformIO) | ST-Link<br>(puce sur la<br>carte NUCLEO) | STM32L476RG<br>(votre programme) |
| Breakpoints                        | <--> commandes GDB                                       | <--> SWD (2 fils)                        | <--> CPU Cortex-M4               |
| CALL STACK                         | JTAG/SWD                                                 | USB                                      | registres SP                     |
| REGISTERS                          | protocole                                                | PC hôte                                  | mémoire SRAM                     |
| VARIABLES                          |                                                          |                                          |                                  |

*Pour les étudiants : Le débogueur ne modifié PAS votre programme. Il utilise des fonctionnalités matérielles du Cortex-M4 (hardware breakpoints, registres de debug) pour observer et contrôler l'exécution sans impact sur le code.*

## 4.1 Configurer platformio.ini pour le débogage

```
[env:nucleo_l476rg]
platform = ststm32
board    = nucleo_l476rg
framework = stm32cube

# -- Parametres de debogage --
debug_tool      = stlink          ; utiliser le ST-Link intégré
upload_protocol = stlink          ; téléversement aussi via ST-Link
debug_build_flags = -O0 -g        ; -O0 : pas d'optimisation -g : symboles de debug
```

### Pourquoi -O0 (zero optimisation) est INDISPENSABLE en débogage ?

Avec les optimisations activées (-O2 ou -Os), le compilateur GCC peut :

- Supprimer des variables locales jugées "inutiles" -> elles disparaissent du panneau VARIABLES
- Réorganiser l'ordre des instructions -> le pas-à-pas ne correspond plus au code source
- Inliner des fonctions -> elles n'apparaissent plus dans le CALL STACK
- Éliminer du code "mort" -> des lignes entières sont sautées

-O0 désactive toutes ces transformations. Le code compilé est plus lent et plus grand, mais il correspond EXACTEMENT à ce que vous avez écrit. -g ajoute les symboles de debug dans le fichier .elf : noms des fonctions, noms des variables, correspondance ligne de code <-> adresse mémoire.

## 4.2 Premier débogage — pas-à-pas dans le clignotant

1. Ouvrir src/main.c

2. Poser un breakpoint sur la ligne HAL\_GPIO\_WritePin(GPIOA, GPIO\_PIN\_5, GPIO\_PIN\_SET) : cliquer dans la marge gauche -> un point rouge apparaît

3. Lancer le débogage : menu Run -> Start Debugging ou touche F5
4. PlatformIO compile en mode debug et téléverse le firmware sur la carte
5. Le programme s'arrête automatiquement au début de main() — c'est normal
6. Appuyer sur F5 (Continue) pour reprendre l'exécution jusqu'au breakpoint défini

Quand le breakpoint est atteint, la ligne est surlignée en jaune et l'exécution est suspendue.

### 4.3 Les panneaux du débogueur — guide détaillé

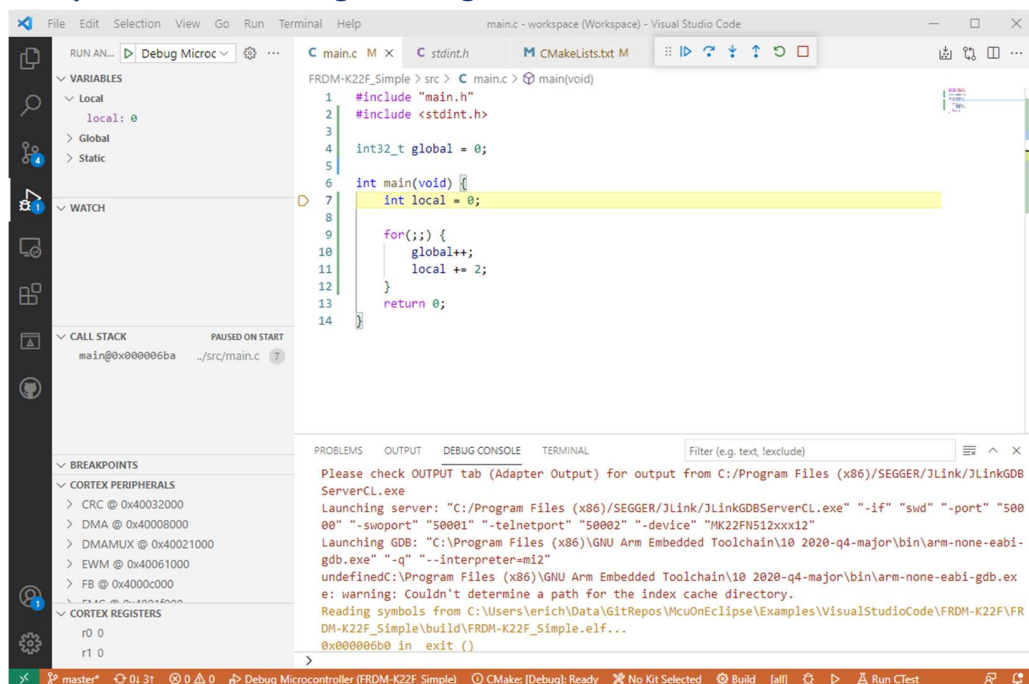


Figure 5 — Session de débogage VSCode avec Cortex-Debug (source : MCU on Eclipse)

### VARIABLES — observer l'état des variables en temps réel

Affiche les variables locales de la fonction courante, les variables globales, et les variables statiques. Leur valeur est mise à jour à chaque pause.

- Local : variables déclarées dans la fonction où l'exécution est arrêtée
- Global : variables déclarées en dehors de toute fonction (dans .data ou .bss)
- Static : variables locales déclarées avec le mot-clé static (persistantes entre les appels)

*Astuce : Pour modifier une variable en direct : double-cliquer sur sa valeur -> saisir une nouvelle valeur -> Entrée. Utile pour tester des cas limites sans recompiler.*

## CALL STACK — voir la pile d'appels active

Affiche la liste des fonctions empilées à l'instant où l'exécution est suspendue. C'est la représentation graphique directe de la pile d'exécution.

- La fonction en haut de la liste = la fonction courante
- Les fonctions en dessous = celles qui l'ont appelée, dans l'ordre
- Cliquer sur un frame = voir ses variables locales dans VARIABLES

*Lien avec la mémoire : Chaque ligne du CALL STACK correspond à une frame sur la pile en SRAM. Le frame du haut (fonction active) est celle qui est au sommet de la pile (adresse la plus basse car la pile grandit vers le bas).*

## REGISTERS — observer les registres du processeur

| Registre | Rôle                                                                                                 |
|----------|------------------------------------------------------------------------------------------------------|
| SP (R13) | Stack Pointer — adresse du sommet de la pile. Diminue à chaque PUSH, augmente à chaque POP.          |
| PC (R15) | Program Counter — adresse de la prochaine instruction à exécuter.                                    |
| LR (R14) | Link Register — adresse de retour, sauvegardée lors d'un appel de fonction (BL).                     |
| R0 - R3  | Registres de paramètres — les 4 premiers paramètres d'une fonction y sont passés (convention AAPCS). |
| R0       | Registre de valeur de retour — la valeur retournée par une fonction est placée dans R0.              |
| xPSR     | Registre d'état — contient les flags N, Z, C, V (negatif, zero, retenue, débordement).               |

## WATCH — surveiller des expressions

- tab[0], tab[1], tab[2] — surveiller des éléments individuels d'un tableau
- \*ptr — voir la valeur pointée par un pointeur
- uwTick — valeur du compteur de millisecondes du HAL

## 4.4 Référence rapide des commandes du débogueur

| Action    | Touche | Description                                              |
|-----------|--------|----------------------------------------------------------|
| Continuer | F5     | Reprend l'exécution jusqu'au prochain breakpoint         |
| Step Over | F10    | Exécute la ligne SANS entrer dans les fonctions appelées |
| Step Into | F11    | ENTRE dans la prochaine                                  |

|                        |               |                                                                            |
|------------------------|---------------|----------------------------------------------------------------------------|
|                        |               | fonction appelée (empile une frame)                                        |
| <b>Step Out</b>        | Shift+F11     | Terminé la fonction courante et revient à l'appelant (dépile le frame)     |
| <b>Arrêter</b>         | Shift+F5      | Arrête le débogage et déconnecte le ST-Link                                |
| <b>Redémarrer</b>      | Ctrl+Shift+F5 | Redémarré le programme depuis Reset_Handler                                |
| <b>Breakpoint</b>      | Clic marge    | Cliquer dans la marge gauche à côté du numéro de ligne                     |
| <b>BP conditionnel</b> | Clic droit    | Sur un breakpoint existant -> "Edit Breakpoint" -> saisir une expression C |

*Conseil pédagogique : Utilisez F11 (Step Into) pour comprendre ce qui se passe quand une fonction est appelée — vous verrez le frame s'empiler dans CALL STACK et SP diminuer dans REGISTERS. Utilisez Shift+F11 (Step Out) pour observer le dépileage.*

## 5 — Exercice : Organisation mémoire

### 5.1 Organisation de la mémoire du STM32L476RG

#### Organisation mémoire du STM32L476RG

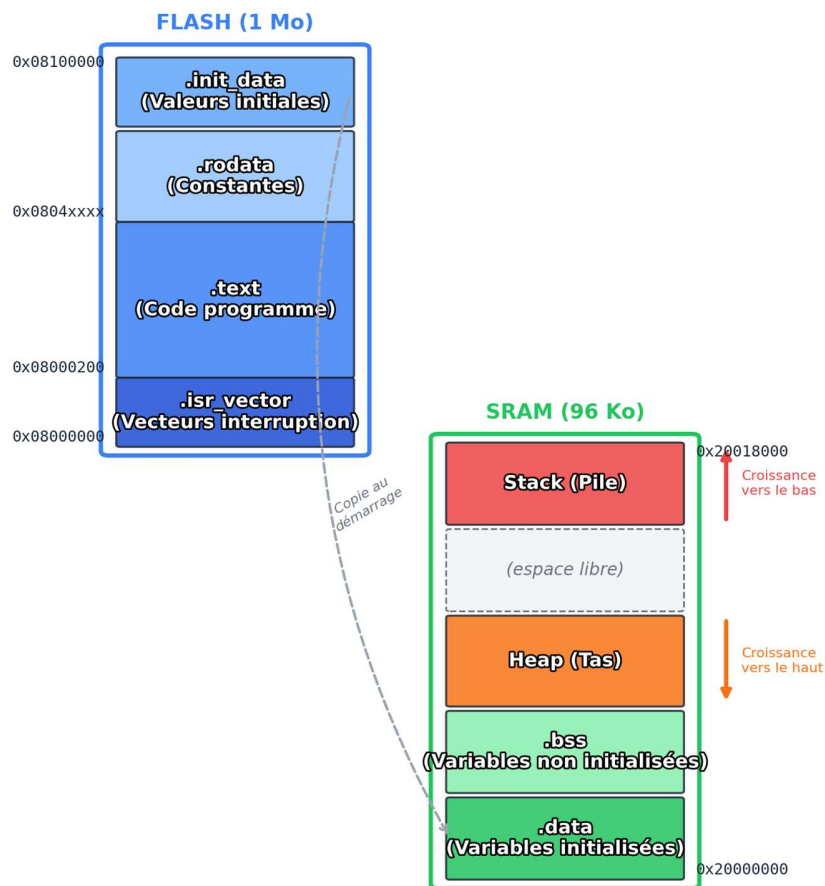


Figure 6 — Organisation mémoire du STM32L476RG (Flash + SRAM)

Quand PlatformIO compile votre programme, le linker (éditeur de liens) répartit le code et les données dans les différentes sections de la mémoire selon un plan précis défini dans le linker script.

```
/* Organisation mémoire du STM32L476RG en cours d'exécution */

FLASH (lecture seule – programme stocke ici)
+-----+ 0x08000000
| Table de vecteurs | adresses des handlers
d'interruption
| .text – code compile (instructions ARM) | vos fonctions + HAL STM32
| .rodata – constantes (const, chaines...) | ex: "Hello\n", valeurs const
| Init de .data (copie au démarrage) | valeurs initiales des globales
+-----+ 0x08100000 (1 Mo)

SRAM1 + SRAM2 (lecture/écriture – données en cours d'exécution)
+-----+ 0x20000000
| .data – variables globales initialisées | ex: int x = 5;
| .bss – variables globales = 0 | ex: int y; (mis à 0 au
démarrage)
|
| Tas (heap) croît vers les adresses + | malloc() – allocation dynamique
|
| ... espace libre ... |
|
| Pile (stack) croît vers les adresses - | variables locales, retours de
fonction
+-----+ 0x20018000 (96 Ko)
<- SP part d'ici (0x20018000) et DIMINUE au fur et à mesure
```

*Explication : Le tas grandit vers le haut (adresses croissantes) depuis la fin de .bss. La pile grandit vers le bas (adresses décroissantes) depuis 0x20018000. Si les deux se rencontrent -> collision = crash (stack overflow ou heap overflow).*

### A retenir — Sections mémoire

- .text et .rodata sont en Flash — ils ne consomment PAS de SRAM.
- .data est en Flash (pour la valeur initiale) ET en SRAM (pour la valeur courante). Au démarrage, Reset\_Handler copie les valeurs initiales de Flash vers SRAM.
- .bss n'est qu'en SRAM — pas de valeur en Flash, il est mis à zéro par le startup code.
- La pile et le tas sont en SRAM — leurs tailles ne sont connues qu'à l'exécution.

## 5.2 Analyser les sections avec arm-none-eabi-size

```
# Se déplacer dans le dossier de build du projet
# Linux :
cd ~/Documents/PlatformIO/Projects/workshop_pile/.pio/build/nucleo_l476rg
# Windows :
# cd
%USERPROFILE%\Documents\PlatformIO\Projects\workshop_pile\.pio\build\nucleo_l476rg

# Affichage résumé (text / data / bss)
```

```
arm-none-eabi-size firmware.elf

# Affichage détaillé section par section
arm-none-eabi-size -A firmware.elf
```

**Exemple de résultat attendu pour le programme LED blink :**

| text  | data | bss | dec   | hex  | filename     |
|-------|------|-----|-------|------|--------------|
| 21080 | 96   | 256 | 21432 | 5388 | firmware.elf |

| Colonne     | Signification                                    | Emplacement mémoire                    |
|-------------|--------------------------------------------------|----------------------------------------|
| <b>text</b> | Taille du code + constantes (.rodata)            | Flash uniquement                       |
| <b>data</b> | Variables globales initialisées (ex: int x = 5)  | Flash (valeur initiale) + SRAM (copie) |
| <b>bss</b>  | Variables globales non initialisées (ex: int y;) | SRAM uniquement, mis à 0 au démarrage  |
| <b>dec</b>  | Total en decimal : text + data + bss             | —                                      |
| <b>hex</b>  | Total en hexadecimal                             | —                                      |

**Question :** Comment interpréter ce résultat ? Quelle est la Flash occupée ? Quelle est la SRAM occupée ?

**Solution :** Flash occupée = text + data = 21 080 + 96 = 21 176 octets, soit environ 2% de 1 Mo.  
 SRAM statique occupée = data + bss = 96 + 256 = 352 octets. SRAM totale en cours d'exécution :  
 il faut ajouter la pile (1 Ko) et le tas (512 octets) = data + bss + pile + tas = 96 + 256 + 1024 + 512 = 1 888 octets sur 98 304 disponibles.

### 5.3 Consulter le linker script

```
/* Tailles minimales de pile et de tas */
_Min_Heap_Size = 0x200; /* 512 octets – espace pour malloc() */
_Min_Stack_Size = 0x400; /* 1024 octets – espace pour les appels de fonction */

/* Plan des régions mémoires */
MEMORY {
  RAM (xrw) : ORIGIN = 0x20000000, LENGTH = 96K /* SRAM totale */
  FLASH (rx) : ORIGIN = 0x08000000, LENGTH = 1024K /* Flash totale */
}
```

*(xrw) signifie : executable, read, write — la SRAM permet les trois. (rx) signifie : read, executable — la Flash est en lecture seule.*

**Question :** Que se passe-t-il si la pile dépasse la taille allouée pendant l'exécution ?

**Solution :** La pile empiète sur le tas ou les variables globales — c'est un "stack overflow". Sur STM32, cela provoque généralement un HardFault. Le programme se bloque dans `while(1){}`. En production, on peut configurer une région MPU (Memory Protection Unit) pour détecter cela automatiquement et réagir proprement.

## 6 — Exercice : Observer la pile en direct

### 6.1 Mesurer la pile depuis le code C

Le registre SP (Stack Pointer) du Cortex-M4 est accessible depuis le code C grâce à l'assembleur inline. Cela permet de mesurer précisément combien d'octets de pile sont utilisés à n'importe quel moment de l'exécution.

Créer un nouveau projet PlatformIO (même procédure que section 3.1, nommer ce projet `workshop_pile_mesure`) et copier ce code dans `src/main.c` :

#### Programme 2 — Mesure de la pile via UART

```
/**
 * Workshop Pile – Programme 2 : Mesure de la pile via UART
 *
 * OBJECTIF PEDAGOGIQUE :
 * Apprendre à lire le registre SP (Stack Pointer) depuis le code C
 * et mesurer la quantité de pile consommée à différents moments.
 *
 * CONNEXION UART :
 * PA2 = TX (STM32 -> PC), PA3 = RX (PC -> STM32)
 * Le ST-Link expose l'UART2 comme port série USB virtuel
 * Linux : /dev/ttyACM0 | Windows : COMx
 */

#include "stm32l4xx_hal.h"
#include <stdio.h>
#include <string.h>

/* Adresse de fin de SRAM = valeur initiale de SP
 * SRAM1 (64 Ko) + SRAM2 (32 Ko) = 96 Ko depuis 0x20000000
 * Fin = 0x20000000 + 0x18000 = 0x20018000 */
#define SRAM_END 0x20018000UL

/* Lire le registre SP via assembleur inline GCC
 * __asm volatile : bloc assembleur non optimisable
 * "MOV %, SP" : copie SP dans le registre %0
 * : "=r" (sp) : %0 = variable C "sp" (registre général)
 * INLINE : pas de frame supplémentaire sur la pile */
```

```
static inline uint32_t get_sp(void) {
    uint32_t sp;
    __asm volatile ("MOV %0, SP" : "=r" (sp));
    return sp;
}

/* Calculer les octets de pile consommés
 * Formule : pile_utilisee = SRAM_END - SP_courant
 * Plus SP est petit, plus la pile est grande */
static inline uint32_t pile_utilisee(void) {
    return SRAM_END - get_sp();
}

void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_USART2_UART_Init(void);

UART_HandleTypeDef huart2; /* Handle global UART */

int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_USART2_UART_Init();

    char msg[80];

    /* Afficher la taille de pile au démarrage
     * %lu = format pour unsigned long (uint32_t sur ARM = 32 bits)
     * \r\n = retour chariot + nouvelle ligne (format terminal série) */
    sprintf(msg, "[main] Pile utilisee : %lu octets\r\n", pile_utilisee());
    HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);

    /* EXERCICE : Décommentez pour mesurer SP directement
     * sprintf(msg, "[main] SP = 0x%08lX\r\n", (unsigned long)get_sp());
     * HAL_UART_Transmit(&huart2, (uint8_t*)msg, strlen(msg), 100);
     */

    while (1) {}
}

/* Configuration horloge – identique Programme 1 (voir section 3.3)
 * HSI 16 MHz -> PLL (div1, x10, div2) -> SYSCLK 80 MHz */
void SystemClock_Config(void) {
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
```

```
RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
RCC_OscInitStruct.OscillatorType      = RCC_OSCILLATORTYPE_HSI;
RCC_OscInitStruct.HSISState           = RCC_HSI_ON;
RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
RCC_OscInitStruct.PLL.PLLState        = RCC_PLL_ON;
RCC_OscInitStruct.PLL.PLLSource       = RCC_PLLSOURCE_HSI;
RCC_OscInitStruct.PLL.PLLM            = 1;
RCC_OscInitStruct.PLL.PLLN            = 10;
RCC_OscInitStruct.PLL.PLLR            = RCC_PLLR_DIV2;
HAL_RCC_OscConfig(&RCC_OscInitStruct);
RCC_ClkInitStruct.ClockType            = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYSCLK
                                         | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
RCC_ClkInitStruct.SYSClockSource       = RCC_SYSCLOCKSOURCE_PLLCLK;
RCC_ClkInitStruct.AHBCLKDivider        = RCC_SYSCLOCK_DIV1;
RCC_ClkInitStruct.APB1CLKDivider       = RCC_HCLK_DIV1;
RCC_ClkInitStruct.APB2CLKDivider       = RCC_HCLK_DIV1;
HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4);
}

/* UART2 : PA2 = TX, PA3 = RX (connecte au ST-Link)
 * Mode Alternate Function : le périphérique USART2 contrôle les broches
 * GPIO_AF7_USART2 = fonction alternative n7 du multiplexeur */
static void MX_USART2_UART_Init(void) {
    __HAL_RCC_USART2_CLK_ENABLE();
    __HAL_RCC_GPIOA_CLK_ENABLE();
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    GPIO_InitStruct.Pin              = GPIO_PIN_2 | GPIO_PIN_3;
    GPIO_InitStruct.Mode              = GPIO_MODE_AF_PP;
    GPIO_InitStruct.Pull              = GPIO_NOPULL;
    GPIO_InitStruct.Speed             = GPIO_SPEED_FREQ_VERY_HIGH;
    GPIO_InitStruct.Alternate         = GPIO_AF7_USART2;
    HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
    huart2.Instance                  = USART2;
    huart2.Init.BaudRate              = 115200;
    huart2.Init.WordLength            = UART_WORDLENGTH_8B;
    huart2.Init.StopBits              = UART_STOPBITS_1;
    huart2.Init.Parity                = UART_PARITY_NONE;
    huart2.Init.Mode                  = UART_MODE_TX_RX;
    HAL_UART_Init(&huart2);
}

/* GPIO : PA5 = LED LD2 */
static void MX_GPIO_Init(void) {
    GPIO_InitTypeDef GPIO_InitStruct = {0};
    __HAL_RCC_GPIOA_CLK_ENABLE();
```

```

GPIO_InitStruct.Pin    = GPIO_PIN_5;
GPIO_InitStruct.Mode   = GPIO_MODE_OUTPUT_PP;
GPIO_InitStruct.Pull   = GPIO_NOPULL;
GPIO_InitStruct.Speed  = GPIO_SPEED_FREQ_LOW;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);
}

/* Handlers d'interruption */
void SysTick_Handler(void) { HAL_IncTick(); }
void NMI_Handler(void)     {}
void HardFault_Handler(void) { while (1) {} }

```

**Lire les messages UART dans VSCode :**

### Communication série UART2 via ST-Link

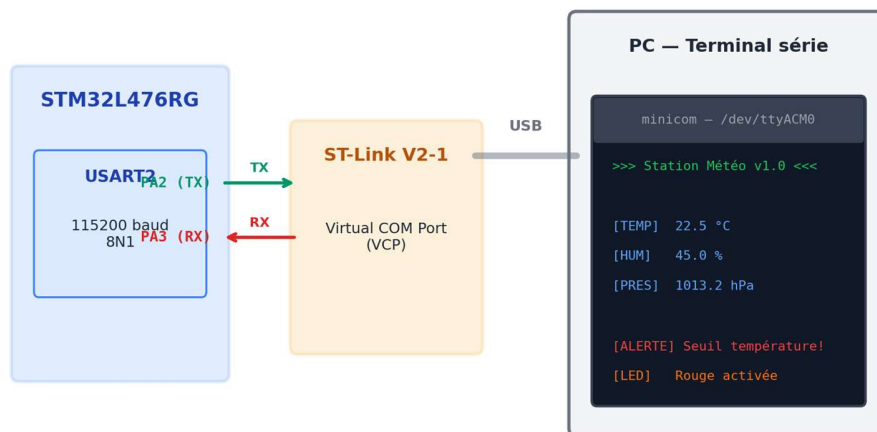


Figure 7 — Communication série UART2 via ST-Link

1. Brancher la carte et téléverser le programme (Ctrl+Alt+U)
2. Cliquer sur l'icône prise électrique en bas de VSCode (Serial Monitor de PlatformIO)
3. Sélectionner le bon port (/dev/ttyACM0 sous Linux ou COM3 sous Windows) et 115200 baud
4. Les messages apparaissent dans la fenêtre

**Question :** A quoi correspond le nombre affiche ? Pourquoi n'est-il pas zéro ?

**Solution :** Il s'agit du nombre d'octets déjà occupés dans la pile lorsque `main()` s'exécute. Ce nombre n'est pas zéro car le code de démarrage du STM32 (`Reset_Handler`) a déjà empile plusieurs frames avant d'appeler `main()` : initialisation des sections `.data` et `.bss`, appel de

HAL\_Init(). La formule `SRAM_END - SP` donne exactement l'espace pile consommé. Valeur typique : entre 100 et 300 octets.

## 6.2 Exercice principal — observer la pile lors d'appels imbriqués

**Objectif :** Observer concrètement comment la pile grandit et rétrécit lors d'appels de fonctions imbriqués, en utilisant le débogueur pas-à-pas.

**Ce que vous allez implémenter :**

- Une fonction `add(int a, int b)` qui additionne deux entiers et retourne le résultat
- Une fonction `sumInt(int n)` qui calcule la somme  $0 + 1 + 2 + \dots + (n-1)$  en appelant `add()` à chaque itération

**Frames empilées : main() → sumInt() → add()**

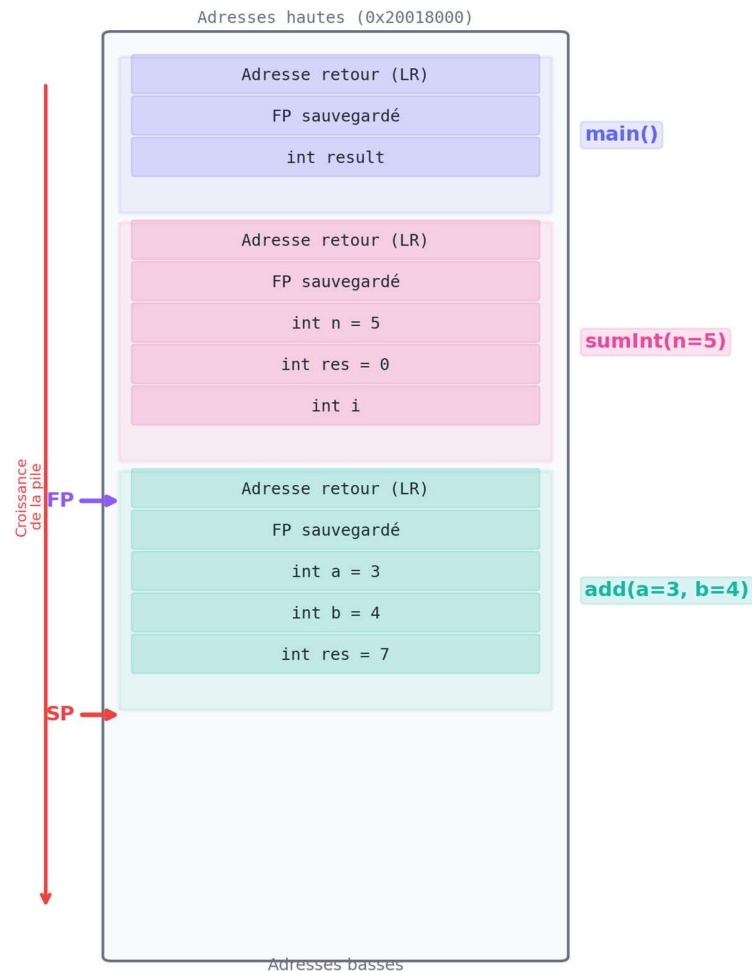


Figure 8 — Frames empilées : main() -> sumInt() -> add()

- Un appel à sumInt(100) depuis main()
- Un breakpoint conditionnel qui se déclenche uniquement à la 90ème itération

**Ce que vous allez observer avec le débogueur :**

- CALL STACK : 3 frames empilées simultanément (main -> sumInt -> add)
- VARIABLES : les variables locales de chaque frame, visibles en cliquant sur chaque frame
- REGISTERS : la valeur de SP diminue quand add() est appelée, remonte quand add() se termine

### Programme 3 — Appels imbriqués (code complet)

```
/**
 * Workshop Pile – Programme 3 : Appels imbriqués
 *
 * OBJECTIF : Observer comment la pile évolue lors d'appels imbriqués.
 *
 * ARCHITECTURE :
 *   main() -> sumInt(100) -> add(res, i) [appelé 100 fois]
 *
 * CE QUE VOUS DEVEZ OBSERVER :
 *   1. CALL STACK : add -> sumInt -> main -> Reset_Handler
 *   2. SP diminue quand on entre dans add(), augmente quand add() retourne
 *   3. Les variables de add() disparaissent quand add() se termine
 */

#include "stm32l4xx_hal.h"

void SystemClock_Config(void);

/* FONCTION add() : additionne a et b
 *
 * CONVENTION D'APPEL ARM AAPCS :
 *   Parametre a -> registre R0
 *   Parametre b -> registre R1
 *   Valeur de retour -> registre R0
 *
 * CE QUI SE PASSE SUR LA PILE (avec -00) :
 *   1. L'instruction BL stocke l'adresse de retour dans LR
 *   2. Le prologue genere par le compilateur :
 *       PUSH {lr}          -> sauvegarde LR (4 octets)
 *       SUB sp, sp, #12 -> reserve 12 octets pour a, b, res
 *   3. Les parametres R0 et R1 sont copies sur la pile (car -00)
 *   4. Le calcul est effectue
 *   5. L'epilogue :
 *       ADD sp, sp, #12 -> libere les 12 octets
 *       POP {pc}          -> restaure LR dans PC = retour
 *
 * TAILLE DE FRAME ESTIMEE : ~16 octets
 *
 * ATTENTION : NE PAS CONFONDRE avec l'empilement d'interruption !
 *   L'empilement automatique de 32 octets (R0-R3, R12, LR, PC, xPSR)
 *   ne se produit que lors d'une INTERRUPTION, PAS lors d'un appel BL. */
int add(int a, int b) {
    int res;          /* variable locale visible dans VARIABLES */
```

```
    res = a + b;    /* POSER UN BREAKPOINT CONDITIONNEL ICI */
    return res;    /* La valeur est copiée dans R0 avant le retour */
}

/* FONCTION sumInt() : somme les n premiers entiers
*
* RESULTAT MATHEMATIQUE : sumInt(100) = 0+1+...+99 = n(n-1)/2 = 4950
*
* COMPORTEMENT DE LA PILE PENDANT LA BOUCLE :
*   A chaque iteration :
*     1. sumInt appelle add() -> frame de add() EMPILEE (SP diminue)
*     2. add() retourne -> frame de add() DEPILEE (SP remonte)
*     3. SP revient a sa valeur d'avant l'appel
*   La pile n'AUGMENTE PAS à chaque itération car add() se termine
*   avant l'itération suivante. Très différent d'une récursion !
*
* VERIFICATION a i=89 (90eme itération) :
*   Somme partielle = 0+1+...+88 = 88*89/2 = 3916
*   -> add(3916, 89) est appele -> a=3916, b=89 */
int sumInt(int n) {
    int res = 0;
    for (int i = 0; i < n; i++) {
        res = add(res, i); /* add() s'empile puis se dépile */
    }
    return res;
}

/* Variable globale volatile pour stocker le résultat
*
* POURQUOI volatile ?
*   Indique au compilateur que cette variable peut être lue/modifiée
*   par un mécanisme externe (débugueur, interruption, périphérique).
*   Le compilateur ne doit JAMAIS :
*     - Supprimer les écritures vers cette variable
*     - Garder sa valeur en cache dans un registre
*     - Réordonner les accès
*
*   Avec -O2, GCC pourrait supprimer l'appel a sumInt() si résultat
*   n'est jamais relu. Volatile empêche cela.
*
* EMPLACEMENT MEMOIRE :
*   Variable GLOBALE -> section .bss (initialisée à 0), PAS sur la pile. */
volatile int resultat;

/* ETAT DE LA PILE A DIFFERENTS MOMENTS :
```

```
* Avant sumInt :      [Reset_Handler] [main]
* Pendant sumInt :    [Reset_Handler] [main] [sumInt]
* Pendant add :       [Reset_Handler] [main] [sumInt] [add]
* Après add retourne : [Reset_Handler] [main] [sumInt]
* Après sumInt :      [Reset_Handler] [main] */
int main(void) {
    HAL_Init();
    SystemClock_Config();

    /* BREAKPOINT 1 : noter la valeur de SP dans REGISTERS */
    resultat = sumInt(100);
    /* BREAKPOINT 2 : resultat = 4950, SP est revenu a sa valeur d'avant */

    while (1) { }
}

/* Configuration horloge – identique Programmes 1 et 2 */
void SystemClock_Config(void) {
    RCC_OscInitTypeDef RCC_OscInitStruct = {0};
    RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};
    RCC_OscInitStruct.OscillatorType      = RCC_OSCILLATORTYPE_HSI;
    RCC_OscInitStruct.HSISState           = RCC_HSI_ON;
    RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
    RCC_OscInitStruct.PLL.PLLState        = RCC_PLL_ON;
    RCC_OscInitStruct.PLL.PLLSource       = RCC_PLLSOURCE_HSI;
    RCC_OscInitStruct.PLL.PLLM            = 1;
    RCC_OscInitStruct.PLL.PLLN            = 10;
    RCC_OscInitStruct.PLL.PLLR            = RCC_PLLR_DIV2;
    HAL_RCC_OscConfig(&RCC_OscInitStruct);
    RCC_ClkInitStruct.ClockType            = RCC_CLOCKTYPE_HCLK | RCC_CLOCKTYPE_SYCLK
                                           | RCC_CLOCKTYPE_PCLK1 | RCC_CLOCKTYPE_PCLK2;
    RCC_ClkInitStruct.SYCLKSource          = RCC_SYCLKSOURCE_PLLCLK;
    RCC_ClkInitStruct.AHBCLKDivider        = RCC_SYCLK_DIV1;
    RCC_ClkInitStruct.APB1CLKDivider       = RCC_HCLK_DIV1;
    RCC_ClkInitStruct.APB2CLKDivider       = RCC_HCLK_DIV1;
    HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_4);
}

/* Handlers d'interruption */
void SysTick_Handler(void) { HAL_IncTick(); }
void NMI_Handler(void)      {}
void HardFault_Handler(void) { while (1) {} }
```

### 6.3 Poser un breakpoint conditionnel

Un breakpoint conditionnel ne suspend l'exécution que lorsqu'une condition C est vraie. C'est beaucoup plus efficace que d'appuyer sur F5 90 fois !

#### Procédure pas-à-pas :

1. Dans add(), cliquer dans la marge gauche à côté de la ligne `res = a + b;` pour poser un breakpoint (point rouge)
2. Faire un clic droit sur ce point rouge -> sélectionner "Edit Breakpoint..."
3. Une boîte de texte apparaît — y taper la condition : `b == 89`
4. Appuyer sur Entrée pour valider — le point rouge devient un point rouge avec un signe égal =
5. Lancer le débogage (F5) — le programme tourne librement jusqu'à ce que b vaille 89

*Pourquoi `b == 89` ? `sumInt()` appelle `add(res, i)` avec `i` allant de 0 à 99. A la 90ème iteration, `i = 89` -> `add(3916, 89)` -> `b = 89`. Le deuxième paramètre `b` de `add()` reçoit la valeur de `i`.*

### 6.4 Ce que vous devez observer quand le breakpoint se déclenche

#### CALL STACK attendu :

```
> add      <- frame active (exécution suspendue sur res = a + b)
sumInt     <- a appelle add(res, i) avec i=89
main       <- a appelle sumInt(100)
Reset_Handler <- code de démarrage STM32
```

*Ce que cela signifie en mémoire : Il y a 4 frames empilées en SRAM. De 0x20018000 (haut) vers les adresses basses : Reset\_Handler, main, sumInt, add. SP pointe vers le bas de la frame de add().*

#### VARIABLES — frame de add() :

| Variable   | Valeur attendue | Explication                                                  |
|------------|-----------------|--------------------------------------------------------------|
| <b>a</b>   | 3916            | Somme partielle $0+1+\dots+88 = 88 \times 89 / 2 = 3916$     |
| <b>b</b>   | 89              | Valeur courante de <code>i</code> dans <code>sumInt</code>   |
| <b>res</b> | Indéterminée    | Pas encore calculée (on est AVANT <code>res = a + b</code> ) |

#### VARIABLES — frame de sumInt() (cliquer sur sumInt dans CALL STACK) :

| Variable   | Valeur attendue | Explication                |
|------------|-----------------|----------------------------|
| <b>n</b>   | 100             | Paramètre passe par main   |
| <b>res</b> | 3916            | Somme accumulée jusqu'ici  |
| <b>i</b>   | 89              | Compteur de boucle courant |

**REGISTERS — observer SP avant et après l'appel :**

1. Poser un breakpoint sur `res = add(res, i);` dans `sumInt()` et noter la valeur de SP
2. Appuyer sur F11 (Step Into) pour entrer dans `add()` et noter la nouvelle valeur de SP
3. La différence = taille de la frame de `add()` empilée

*Exemple : Si SP = 0x2001FF80 dans `sumInt` et SP = 0x2001FF70 dans `add`, alors la frame de `add` fait  $0x2001FF80 - 0x2001FF70 = 16$  octets.*

**Questions de validation**

**Question :** Question 1 — Combien de frames sont simultanément actives quand `add()` est en cours d'exécution ?

**Solution :** 4 frames actives : `Reset_Handler()` -> `main()` -> `sumInt()` -> `add()`. La frame de `add()` contient a (4 octets), b (4 octets), res (4 octets) = 12 octets de variables locales, plus LR sauvegardé (4 octets). La frame fait donc ~16 octets avec -O0.

**Question :** Question 2 — Que se passe-t-il dans le CALL STACK quand on sort de `add()` avec Step Out (Shift+F11) ?

**Solution :** La frame de `add()` est dépilée — SP remonte. Le CALL STACK ne montre plus `add()` — seulement `sumInt()` et `main()`. La valeur retournée par `add()` ( $3916 + 89 = 4005$ ) a été placée dans le registre R0 (convention ARM AAPCS). `sumInt()` la récupère depuis R0 et la stocke dans sa variable locale res.

**Question :** Question 3 — Quelle est la différence entre observer SP dans REGISTERS et observer la pile dans CALL STACK ?

**Solution :** SP dans REGISTERS donne l'adresse brute du sommet de pile en hexadecimal (ex : 0x2001FF20). CALL STACK donne une vue symbolique : noms des fonctions, ordre des appels, variables locales. Les deux sont complémentaires — SP mesure l'occupation en octets, CALL STACK explique le contenu.

**Question :** Question 4 — Pourquoi utilise-t-on volatile pour la variable "résultat" dans `main()` ?

**Solution :** Le mot-clé volatile indique au compilateur que la variable peut être modifiée par un mécanisme externe. Avec les optimisations activées (-O2), GCC pourrait remarquer que résultat

n'est jamais relu et supprimer complètement l'appel à `sumInt()`. `volatile` empêche cela. Avec -O0, c'est peu probable mais `volatile` reste une bonne pratique en embarqué.

**Question :** Question 5 (bonus) — Que se passerait-il si `sumInt()` était récursive au lieu d'itérative ?

**Solution :** Chaque appel récursif empilerait un nouveau frame SANS dépiler la précédente. Pour `sumInt(100)`, il y aurait 100 frames simultanément sur la pile. Avec ~20-30 octets par frame, cela consommerait ~2-3 Ko — plus que les 1 Ko par défaut ! Avec `sumInt(10000)`, on provoquerait un stack overflow et un `HardFault`. C'est pourquoi la version itérative (boucle `for`) est préférée en embarqué.

**Question :** Question 6 — Lien avec le prosit : dans l'algorithme `Nouveau_Algo_Mesure`, la chaîne d'appels la plus profonde est `main()` → `lecture()` → `add_val()`. Combien de frames sont empilées simultanément ? Estimez la taille totale de la pile au pic.

Réponse : 3 frames empilées simultanément. Frame `main()` ≈ 36 octets (`nb_erreur[5]` + `moy_gliss[10]` + LR), frame `lecture()` ≈ 24 octets (2 pointeurs + 3 locales + LR), frame `add_val()` ≈ 16 octets (1 pointeur + val + LR). Total estimé ≈ 76 octets au pic, soit ~7% de la pile par défaut (1024 octets).

## Fin du workshop

*Lien avec le prosit : Les techniques d'observation de la pile que vous avez pratiquées dans ce workshop s'appliquent directement à l'algorithme Nouveau\_Algo\_Mesure du prosit. Utilisez le débogueur pour placer un breakpoint dans add\_val() et observer les 3 frames empilées (main → lecture → add\_val). Comparez la taille de pile mesurée avec votre calcul théorique.*

Vous avez configuré un environnement de développement professionnel pour STM32, compilé et débogué votre premier programme C avec le débogueur natif ST-Link, analysé la mémoire de votre firmware, et observé en temps réel le comportement de la pile lors d'appels de fonctions imbriqués.

### Pour aller plus loin :

- Modifier le programme pour mesurer la pile a plusieurs points (avant/après chaque appel de fonction)
- Ajouter une variable locale de type tableau (int tab[20]) dans add() et observer l'impact sur SP — la frame devrait grandir de 80 octets (20 x 4)
- Consulter le manuel PM0214 (Cortex-M4 Programming Manual, chapitre 2) pour comprendre le mécanisme d'empilement automatique du Cortex-M4 lors des interruptions
- Expérimenter avec -O2 : relancer le débogage avec les optimisations activées et constater que les variables disparaissent et que des fonctions sont inlinées
- Compléter la Corbeille d'exercices — Débogage LED & UART pour pratiquer le débogage sur un projet de station météo embarquée avec LED RGB et communication série